Créer un site web avec des API : guide étape par étape

Nov 11, 2026 · @Michel

Vue d'ensemble

Ce guide montre comment construire un site web simple qui va chercher des données sur une API externe, les affiche dynamiquement, et permet à l'utilisateur de les chercher et de les filtrer grâce à des champs de saisie (inputs).

Exemple concret utilisé tout au long du guide : un site qui affiche une liste de pays, avec un champ de recherche par nom et un filtre par région.

Prérequis

- Bases en HTML (balises, structure d'une page)
- Bases en CSS (sélecteurs, mise en forme)
- Bases en JavaScript (variables, fonctions, `fetch`, manipulation du DOM)

Les 6 étapes du guide

1. Structure HTML de la page
2. Les inputs (recherche, filtre)
3. Appel de l'API avec `fetch`
4. Logique de filtrage des résultats
5. Mise en forme avec CSS
6. Exemple complet assemblé

Étape 1 — Structure HTML de base

On commence par le squelette de la page : un titre, une zone de recherche/filtres, et une zone où les résultats de l'API seront affichés.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Explorateur de pays</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <h1>Explorateur de pays</h1>
```

```
  <!-- Zone des inputs / filtres -->
  <div class="controles">
    <input type="text" id="recherche" placeholder="Rechercher un pays..." />
    <select id="filtre-region">
      <option value="">Toutes les régions</option>
      <option value="Europe">Europe</option>
      <option value="Asia">Asie</option>
      <option value="Africa">Afrique</option>
      <option value="Americas">Amériques</option>
    </select>
  </div>

  <!-- Zone où les résultats seront injectés par JavaScript -->
  <div id="resultats"></div>

  <script src="script.js"></script>
</body>
</html>
```

Trois fichiers séparés, c'est la bonne pratique :

- `index.html` — la structure
- `style.css` — l'apparence
- `script.js` — le comportement (inputs, appel API, filtres)

Étape 2 — Les inputs

Les inputs sont les éléments avec lesquels l'utilisateur interagit : champ de texte, menu déroulant, cases à cocher, etc. L'objectif ici est de récupérer leur valeur en JavaScript à chaque changement.

Récupérer les éléments dans le DOM

```
const inputRecherche = document.getElementById("recherche");
const selectRegion = document.getElementById("filtre-region");
const zoneResultats = document.getElementById("resultats");
```

Écouter les événements

On utilise `addEventListener` pour réagir à chaque frappe ou changement de sélection :

```
// "input" se déclenche à chaque caractère tapé
inputRecherche.addEventListener("input", () => {
  console.log("Valeur tapée :", inputRecherche.value);
  appliquerFiltres();
});

// "change" se déclenche quand on change l'option sélectionnée
selectRegion.addEventListener("change", () => {
  console.log("Région choisie :", selectRegion.value);
  appliquerFiltres();
});
```

Types d'inputs courants

Type	Usage	Exemple
`text`	Recherche, saisie libre	`<input type="text">`
`number`	Valeur numérique (prix min/max)	`<input type="number">`
`checkbox`	Option oui/non	`<input type="checkbox">`
`radio`	Un choix parmi plusieurs	`<input type="radio">`
`select`	Liste déroulante	`<select><option>`
`range`	Curseur (ex. budget max)	`<input type="range">`

Astuce : éviter trop d'appels pendant la frappe (debounce)

Si chaque frappe déclenche un appel API, on peut vite en envoyer trop. On retarde l'exécution avec un `setTimeout` qu'on annule à chaque nouvelle frappe :

```
let minuteur;
inputRecherche.addEventListener("input", () => {
  clearTimeout(minuteur);
  minuteur = setTimeout(() => appliquerFiltres(), 300); // attend 300ms de
pause
});
```

Étape 3 — Appeler l'API

On récupère les données une seule fois au chargement de la page, puis on les garde en mémoire pour les filtrer localement (plus rapide que de refaire un appel API à chaque frappe).

```
let tousLesPays = []; // mémoire locale des données reçues

async function chargerPays() {
  try {
    const reponse = await fetch("https://restcountries.com/v3.1/all?
fields=name,region,capital,flags");
    if (!reponse.ok) throw new Error(`Erreur HTTP : ${reponse.status}`);

    tousLesPays = await reponse.json();
    afficherResultats(tousLesPays); // affichage initial, sans filtre
  } catch (erreur) {
    zoneResultats.innerHTML = `<p>Impossible de charger les données :
${erreur.message}</p>`;
  }
}

// On lance le chargement dès que la page est prête
chargerPays();
```

Afficher les résultats dans le DOM

```
function afficherResultats(pays) {
  if (pays.length === 0) {
    zoneResultats.innerHTML = "<p>Aucun résultat.</p>";
    return;
  }

  zoneResultats.innerHTML = pays
    .map(
      (p) => `
      <div class="carte-pays">
        <img src="${p.flags.png}" alt="Drapeau ${p.name.common}" />
        <h3>${p.name.common}</h3>
        <p>Région : ${p.region}</p>
        <p>Capitale : ${p.capital ? p.capital[0] : "N/A"}</p>
      </div>`
    )
    .join("");
}
```

Points importants

- `fetch` renvoie une `Promise` : on utilise `async` / `await` pour attendre la réponse sans bloquer la page.

- On génère le HTML des résultats avec `.map()` puis `.join("")`, et on l'injecte avec `innerHTML`.
- On garde les données brutes (`tousLesPays`) en mémoire : les filtres (étape suivante) travaillent sur cette copie locale, sans redemander l'API.

Étape 4 — Les filtres

La fonction `appliquerFiltres()` lit la valeur actuelle des inputs, filtre le tableau `tousLesPays` gardé en mémoire, puis réaffiche seulement les résultats correspondants.

```
function appliquerFiltres() {
  const texte = inputRecherche.value.toLowerCase().trim();
  const region = selectRegion.value;

  const resultatsFiltres = tousLesPays.filter((pays) => {
    const correspondNom = pays.name.common.toLowerCase().includes(texte);
    const correspondRegion = region === "" || pays.region === region;

    // Les deux conditions doivent être vraies pour garder le pays
    return correspondNom && correspondRegion;
  });

  afficherResultats(resultatsFiltres);
}
```

Logique à retenir

- `.filter()` parcourt un tableau et garde uniquement les éléments pour lesquels la fonction retourne `true`.
- Combiner plusieurs filtres = combiner plusieurs conditions avec `&&` (ET logique) : toutes les conditions doivent être vraies.
- Un filtre "vide" (ex. région non choisie) doit laisser passer tout le monde : d'où la condition `region === "" || ...`.
- `.toLowerCase()` rend la recherche insensible à la casse ("France" trouve aussi "france").

Ajouter un filtre supplémentaire (exemple : tri par nom)

```
function trierParNom(pays) {
  return [...pays].sort((a, b) =>
a.name.common.localeCompare(b.name.common));
}

// Dans appliquerFiltres(), juste avant afficherResultats :
afficherResultats(trierParNom(resultatsFiltres));
```

Chaque nouveau critère (prix, date, catégorie…) suit le même principe : un input qui déclenche `appliquerFiltres()`, et une condition supplémentaire dans le `.filter()`.

Étape 5 — Le CSS

Le CSS s'occupe de la mise en page générale, du style des inputs/filtres, et de l'affichage des résultats en grille responsive.

```
/* Mise en page générale */
body {
  font-family: system-ui, sans-serif;
  max-width: 1000px;
  margin: 0 auto;
  padding: 1.5rem;
  background-color: #f7f7f9;
  color: #1a1a1a;
}

h1 {
  text-align: center;
  margin-bottom: 1.5rem;
}

/* Barre de contrôles : input + select côte à côte */
.controles {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;           /* passe à la ligne sur petit écran */
  margin-bottom: 2rem;
}

#recherche,
#filtre-region {
  padding: 0.6rem 1rem;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 1rem;
```

```
}

#recherche {
  flex: 1;                    /* prend tout l'espace disponible */
  min-width: 200px;
}

#recherche:focus,
#filtre-region:focus {
  outline: none;
  border-color: #4a7dfc;
  box-shadow: 0 0 0 3px rgba(74, 125, 252, 0.2);
}

/* Grille des résultats, responsive */
#resultats {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 1rem;
}

.carte-pays {
  background: white;
  border-radius: 12px;
  padding: 1rem;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.08);
  transition: transform 0.15s ease;
}

.carte-pays:hover {
  transform: translateY(-3px);
}

.carte-pays img {
  width: 100%;
  height: 110px;
  object-fit: cover;
  border-radius: 8px;
  margin-bottom: 0.5rem;
}

.carte-pays h3 {
  margin: 0.3rem 0;
  font-size: 1.1rem;
}
```

```
/* Responsive : sur mobile, les contrôles s'empilent */
@media (max-width: 480px) {
  .controles {
    flex-direction: column;
  }
}
```

Points clés CSS

- `display: flex` pour aligner l'input et le select horizontalement ; `flex-wrap` pour qu'ils passent à la ligne sur petit écran.
- `display: grid` avec `auto-fill` + `minmax()` crée une grille responsive sans media query supplémentaire : le nombre de colonnes s'ajuste automatiquement à la largeur de l'écran.
- `:focus` améliore l'accessibilité en montrant clairement quel champ est actif.
- `@media (max-width: ...)` adapte la mise en page aux petits écrans (mobile).

Étape 6 — Exemple complet assemblé

Le fichier `script.js` complet, qui regroupe les étapes 2 à 4 (inputs + appel API + filtres) :

```
// --- Références aux éléments du DOM ---
const inputRecherche = document.getElementById("recherche");
const selectRegion = document.getElementById("filtre-region");
const zoneResultats = document.getElementById("resultats");

let tousLesPays = [];

// --- Appel de l'API au chargement de la page ---
async function chargerPays() {
  try {
    const reponse = await fetch("https://restcountries.com/v3.1/all?
fields=name,region,capital,flags");
    if (!reponse.ok) throw new Error(`Erreur HTTP : ${reponse.status}`);

    tousLesPays = await reponse.json();
    afficherResultats(tousLesPays);
  } catch (erreur) {
    zoneResultats.innerHTML = `<p>Impossible de charger les données :
${erreur.message}</p>`;
  }
}

// --- Affichage des résultats dans le DOM ---
```

```
function afficherResultats(pays) {
  if (pays.length === 0) {
    zoneResultats.innerHTML = "<p>Aucun résultat.</p>";
    return;
  }

  zoneResultats.innerHTML = pays
    .map(
      (p) => `
      <div class="carte-pays">
        <img src="${p.flags.png}" alt="Drapeau ${p.name.common}" />
        <h3>${p.name.common}</h3>
        <p>Région : ${p.region}</p>
        <p>Capitale : ${p.capital ? p.capital[0] : "N/A"}</p>
      </div>`
    )
    .join("");
}

// --- Filtrage selon les valeurs des inputs ---
function appliquerFiltres() {
  const texte = inputRecherche.value.toLowerCase().trim();
  const region = selectRegion.value;

  const resultatsFiltres = tousLesPays.filter((pays) => {
    const correspondNom = pays.name.common.toLowerCase().includes(texte);
    const correspondRegion = region === "" || pays.region === region;
    return correspondNom && correspondRegion;
  });

  afficherResultats(resultatsFiltres);
}

// --- Écoute des événements, avec un léger délai sur la recherche ---
let minuteur;
inputRecherche.addEventListener("input", () => {
  clearTimeout(minuteur);
  minuteur = setTimeout(appliquerFiltres, 300);
});

selectRegion.addEventListener("change", appliquerFiltres);

// --- Démarrage ---
chargerPays();
```

Avec le `index.html` de l'étape 1, le `style.css` de l'étape 5 et ce `script.js`, le site fonctionne de bout en bout : chargement des données depuis l'API, affichage en grille, recherche en temps réel et filtre par région, le tout avec une mise en forme responsive.

Pour aller plus loin

- [ ] Gérer un état de chargement (afficher "Chargement..." pendant l'appel API)
- [ ] Ajouter plusieurs filtres combinés (région + population + langue)
- [ ] Paginer les résultats si la liste est longue
- [ ] Stocker les derniers filtres choisis dans `localStorage` pour les retrouver au prochain chargement
- [ ] Passer à un framework (React, Vue) une fois les bases maîtrisées, pour les sites plus complexes
- [ ] Ajouter la gestion des erreurs réseau (pas d'internet, API indisponible) avec un message clair pour l'utilisateur

Bonnes pratiques à garder en tête

- Toujours vérifier `reponse.ok` après un `fetch`, une erreur HTTP ne déclenche pas automatiquement un `catch`.
- Filtrer les données côté client (une fois chargées) est plus rapide que de refaire un appel API à chaque frappe, tant que le volume de données reste raisonnable.
- Séparer HTML / CSS / JS dans des fichiers distincts facilite la maintenance.